# Moderated Learning Management System

Final report, CS4433 Software Engineering

Lebohang Lebete (202321241) | Supervisor: Dr Mamoqenelo Morolong

**Abstract.** This report follows the Moderated Learning Management System from planning to the v1.1 maintenance release. The system lets a school accept assignments, mark them, moderate the marks and release them to students without an internet connection. A console client and a Windows desktop client talk to one on-premises server, which owns the SQLite database and the submission files. The automated tests pass, with one measured exception: login at the 95th percentile took 2.55 seconds against a 2 second requirement. That failure is still open. So are the supervisor's Gate 3 review, a usability trial with real users, and publication to the remote repository.

## 1. Introduction

The school has old computers and an unreliable connection, and it needs a dependable way to run assignments. Storing files is the easy part. The difficult part is getting students, lecturers and heads of department to agree on when a mark counts. The system therefore has to accept work, apply the deadline rule the same way every time, keep written feedback, and show a student a result only after it has been approved. Users, departments, courses, teaching assignments and enrolments are in the system only to support that flow. Payment, automated grading, plagiarism detection, video meetings and web or mobile interfaces were excluded by the brief.

I followed the Waterfall sequence with V-Model verification over seven scheduled work packages. The first three produced the plan, the SRS and the design, and Gate 2 was approved before any code was written. The last four covered implementation, testing, deployment and one change request. The weekly labels describe the order of work, not elapsed calendar time, and no review dates or signatures have been invented. For every result in this report I say whether it was measured, designed or only intended, because a passing build says nothing about usability, legal compliance, or how the system behaves on the school's own hardware.

## 2. Planning and requirements

The feasibility study found nothing that blocked the project. C++17, SQLite, cpp-httplib and nlohmann::json are free and run on a single school machine. The biggest schedule worry was system design, which carries 24 marks but only one to two weeks. The risk register began with eight risks and now has thirteen entries. The two highest exposures are now R10 (the password-hash queue pushing login past its limit, exposure 16) and R6 (a single developer, 15). R10 was added during design and later confirmed by the load test.

Five stakeholders were interviewed, and their statements disagreed in three places. Table 1 shows how each was settled. Two further gaps were closed by assumption: a mark cannot be released without written feedback, and a student may resubmit only while the work is unmarked and before the effective deadline.

The SRS has 28 functional and 13 non-functional requirements, each with a MoSCoW priority, a source and a verification method. The non-functional ones carry numbers so that they can fail: login within 2 seconds at the 95th percentile with 50 concurrent users, 800 submissions in ten minutes, Argon2id password storage and 70% line coverage of the business logic. The only requirement not built is FR28, the notice of newly released marks at login, which was a Could.

*Table 1. Stakeholder conflicts and how the design resolves them.*

| Conflict | Resolution |
|---|---|
| The Principal wants marks hidden until the Head of Department approves them. The Lecturer wants them visible as soon as they are entered. | The Principal's rule wins. Status runs Submitted or Late, Marked, Moderated, Released, and a student sees a mark only at Released. Lecturers can still track moderation status. |
| The Lecturer wants discretion over extensions. The Head of Department wants a strict, automatic penalty. | An extension changes the deadline and nothing else. The penalty is always computed against whichever deadline is in force. |
| Work over three days late gets zero, yet the Student Representative does not want late work rejected. | Acceptance and scoring are separate. Late work is always accepted, marked and given feedback, and its number may be zero. |

## 3. Design

The architecture is a layered client-server system. One server process owns the SQLite file and the submission folder, and clients reach it over HTTP and JSON on the school network. They never open the database. I rejected a shared SQLite file on a network drive because file locking is unreliable there and every PC would hold every password hash. A single-computer MVC program was rejected because 800 students cannot share one console, and a web or microservice design is out of scope and too heavy for a small school.

The server has three layers: API, business logic and data access. Services check authorisation in two steps. First the role must permit the action, through *isPermitted* on one of four User subclasses built by a factory. Then the record must be in the user's scope, meaning an assigned course, their own department or their own submission. The late penalty sits behind a Strategy interface, *IPenaltyPolicy*, so marking and moderation share a single rule, and *SubmissionLifecycle* decides which status changes are legal. All SQL lives in the repositories, which also lets the services run in tests against fakes.

The schema has ten tables in third normal form, with the four roles held in one *app_user* table distinguished by a role column. There is one deliberate exception: *final_mark* and *days_late* are stored as snapshots, so a released mark cannot change when a deadline or policy changes later. The audit table is append-only, enforced by triggers. Designing in detail raised three points for the Gate 2 review. DA1: refuse an extension once work is marked. DA2: accept plain HTTP for v1.0 on an isolated network. DA3: allow fractional marks and calculate the penalty before any rounding.

## 4. Implementation

The first-party code is about 1,400 lines (1,337 in v1.0 and 1,397 in v1.1, counted with Lizard, vendor code and tests excluded). It is split into domain, service, repository, API and client files and built with CMake.

Lateness is rounded up to whole days, so exactly 24 hours late is one day and 24 hours plus one second is two. Days one to three each cost 10% of the assignment maximum, and beyond three days the final mark is zero. A raw mark of 40 out of 50, submitted 1 day 2 hours late, gives d = 2, a penalty of 10 and a final mark of 30. No endpoint accepts a penalty or a final mark from a client. Until release, a student's response simply contains no raw mark, final mark or feedback, instead of carrying hidden fields a modified client could read.

Passwords use libsodium Argon2id at interactive settings with a fresh salt for each user. Each hash needs 64 MiB, so a condition-variable gate allows two at a time, which caps hashing memory at 128 MiB. Sessions are random 256-bit tokens held in memory and expire after 15 idle minutes. Five failed logins lock an account for five minutes, and the lock is stored in the database so it survives a restart. Uploads arrive as multipart data, must be between one byte and 5 MiB, and are stored under

random names with their SHA-256 recorded. A failed transaction deletes its file. A binary download cannot sit safely in a JSON string, so downloads are hex-encoded and decoded by the client. That doubles the download size, which I accepted for files of 5 MB.

Two departures from the approved design were deliberate, and I will need to defend them in the viva. The narrow repository interfaces became one *IRepository* with fixed query and command identifiers, which costs some interface segregation. And one recursively locked SQLite connection serialises reads and writes, replacing the proposed read pool. Both are simpler to get right, and both are recorded as deviations instead of being passed off as the original design. After the main report I also added a native Win32 client, *lms_gui.exe*. The server, database and rules are untouched. Errors show on the same form and keep what the user typed, and drafts survive navigation until sign-out.

## 5. Verification and validation

Tests follow the V-Model: unit tests for the penalty rule, roles and lifecycle; integration tests across services, SQLite and file storage; and system tests that start a real HTTP server and run the use-case workflow. Time rules use a fixed clock and are tested one second before, at and after each boundary. Other cases include twenty SQL-injection strings passed as credentials (none authenticated or changed a row), thirty invalid-input requests, a file exactly at and one byte over the limit, a process kill followed by restart, a writer killed mid-transaction, and a backup restored to a new location.

*Table 2. Automated test evidence.*

| Measure | v1.0 | v1.1 |
|---|---|---|
| Catch2 cases | 14 (219 assertions) | 16 |
| HTTP and system checks | 77 | 81 |
| Business-layer line coverage | 95.77% (475 of 496 lines) | 96.07% (513 of 534 lines) |

*Table 3. Load test against the non-functional targets.*

| Load test (850 accounts, 50 clients) | Requirement | Result |
|---|---|---|
| Login, 95th percentile | 2 s | 2.5528 s, fails |
| Listing, 95th percentile | 2 s | 0.0091 s, passes |
| 800 submissions | 10 min, none corrupted | 44.342 s, 800 checksums verified, integrity ok |

Coverage is measured with gcov over *domain.cpp* and *services.cpp* only, so it says nothing about the whole application, and a high figure does not replace the boundary and authorisation tests. The GUI form checks (28 of 28 passing) and the 90 HTTP and console checks run after the GUI work also pass. Five defects were logged and retested, among them a scalar JSON binding bug, binary data in a JSON field, and a global login mutex that would have serialised unrelated accounts.

The login result is the one real failure. The workload ran on my workstation over loopback, not on the specified two-core, 4 GB profile, so it is evidence from one machine and not a ruling on school hardware. The likely cause is the two-at-a-time hashing gate that protects memory. Lowering the Argon2id settings would make the test pass, but it would reverse an approved security decision, so I have not done it. The next step is to profile hash latency and queue delay on the target server and test a reviewed scheduling change inside the 300 MB limit. Until then the honest conclusion is partial compliance with the performance requirement.

## 6. Configuration management and deployment

The project has a local Git repository with a v1.0 baseline tag and a separate v1.1 change. The supplied remote (TenPlus1-Eng/cs4433-final-project) was empty, and the connected account had read-only access, so nothing has been published. A Git bundle, a source package and a push script hold the history until I can sign in with write access. A GitHub Actions workflow is written but has never run, and I cite no CI result. Cppcheck found two uninitialised members, which I fixed, and a rerun was clean for that configuration. The optimised GCC 16 build warns about a string over-read inside the vendored SQLite 3.45.1 file. I have recorded it and not shown it to be harmless. The code review so far is a self-review, not an independent one.

The Windows x64 package contains both executables, the schema, supporting DLLs, licence notices and four role manuals. The server binds to loopback by default, and a technician may choose a school network address. Backups use the SQLite backup API and copy every referenced submission file, replaced versions included, and the server must be stopped first so that the two stay consistent. Restoring goes to a new database path and file folder and is followed by an integrity check. All times are UTC, entered as YYYY-MM-DDTHH:MM:SSZ. Lesotho is UTC+02:00, so a deadline of local noon is 10:00:00Z, and the manuals say so.

## 7. Maintenance: class-wide extensions (CR1)

CR1 asked for lecturers to extend a deadline for a whole class and for students to see extensions in their assignment list. I classed it as perfective maintenance, since it extends a working feature to a new need and fixes no defect. The effective deadline became the latest of the base, class and individual deadlines. A shorter class extension therefore never removes a longer personal one, and students enrolled later receive the extension too. It is stored once per assignment in a new *class_extension* table, which avoids copying hundreds of individual records and keeps the original deadline visible.

The whole operation is refused if any submission for the assignment is already marked. That keeps DA1 and leaves marked snapshots alone, but it is a policy choice the school should review if it needs extensions after marking. The migration is an idempotent *CREATE TABLE IF NOT EXISTS*. To test it I created a database with the real v1.0 executable, opened it with v1.1, and confirmed that the account record was unchanged, the new table existed, integrity was ok and a second migration run succeeded. Two new test cases and one HTTP check cover precedence, visibility and atomic refusal, and the original suites were rerun and pass.

## 8. Professional, legal and ethical issues

Marks, files and feedback are learner data. Lesotho's Data Protection Act is Act 5 of 2012, although its text gives the short title as 2011, and the school should confirm its obligations with its responsible officer before real use. Role scope, deactivation instead of deletion and the audit trail are sensible safeguards but do not establish compliance. The audit triggers also cannot stop an operating-system administrator who edits the database directly. Plain HTTP means anyone able to capture traffic on the network could read passwords and marks, so TLS has to come before any use on a shared network.

The penalty rule is applied consistently, but consistency is not the same as fairness. One second late counts as a full day, and work more than 72 hours late scores zero even when it is good. The system still accepts that work for feedback. Staff must publish the time convention and run an extension process that can be held to account for illness, disability or disruption. Accessibility is unproven: structured JSON output and UTC input may trip up first-time or screen-reader users in the console, and the GUI should help, but nobody other than me has used either yet.

## 9. Conclusion

The project has a working, tested system that keeps role separation, deadline and penalty rules, moderated release and the audit trail on the server, and that upgrades an existing database without losing data. Four things remain. The login latency has to be fixed or formally accepted after profiling on target hardware. A three-person usability trial and a check on the school's machines have not been done. The supervisor's Gate 3 review has not happened. And the code has to be pushed to the remote repository so that a hosted CI run exists. I have left them visible because they are part of the engineering result.

## References

Libsodium (n.d.) *Password hashing*. libsodium.gitbook.io/doc/password_hashing (accessed 25 October 2026).

SQLite (n.d.) *Write-Ahead Logging*. sqlite.org/wal.html (accessed 25 October 2026).

Lesotho (2012) *Data Protection Act, Act 5 of 2012*. lesotholii.org/akn/ls/act/2012/5/eng@2012-02-22 (accessed 25 October 2026).

Hirose, Y. (2024) *cpp-httplib v0.18.1*; Lohmann, N. (2023) *JSON for Modern C++ v3.11.3*; Catch2 contributors (2022) *Catch2 v2.13.10*. Vendored, with licences.